

Instituto Politécnico Nacional — Escuela Superior de Cómputo

Ingeniería en Sistemas Computacionales / Unidad de aprendizaje: Bases de Datos

Práctica 1 — Ejercicio 3: ¿Qué es una base de datos?

José Luis Villanueva Morales — Boleta 2024630016

1. Dato, información y base de datos

Un dato es la unidad más elemental de representación: un símbolo, número o hecho aislado que, por sí mismo, carece de un significado interpretable en un contexto de decisión. La información, en cambio, es el resultado de procesar, organizar o relacionar datos de manera que adquieran significado y utilidad para quien los recibe (Coronel & Morris, 2019). Elmasri y Navathe (2017) definen una base de datos como una colección de datos relacionados entre sí, que representan algún aspecto del mundo real, están organizados con un propósito específico y son diseñados, contruidos y poblados con datos para una aplicación determinada. Date (2004), por su parte, la describe de forma más operativa como un depósito de datos persistentes usado por los sistemas de aplicación de alguna empresa dada, entendiendo «empresa» como cualquier organización que necesite guardar y utilizar datos para su funcionamiento.

Ambas definiciones coinciden en dos rasgos esenciales —persistencia y organización con un fin — pero difieren en el énfasis: Elmasri y Navathe subrayan la representación de una porción del mundo real (el aspecto semántico), mientras que Date pone el acento en el uso compartido por múltiples programas de aplicación (el aspecto operativo). En conjunto, ambas visiones explican por qué una base de datos no es simplemente «muchos datos juntos», sino una estructura pensada para representar una realidad y servir a las necesidades de información de una organización.

2. Características de una base de datos

Una base de datos bien diseñada exhibe un conjunto de propiedades que la distinguen de una simple colección de archivos (Silberschatz et al., 2020; Coronel & Morris, 2019):

Integración: los datos de distintas áreas de la organización se concentran en una estructura común en lugar de mantenerse en archivos aislados por departamento, lo que evita islas de información.

Persistencia: los datos permanecen almacenados más allá de la ejecución del programa que los creó o modificó; su ciclo de vida no depende de un proceso en particular.

Redundancia controlada: a diferencia de los sistemas de archivos, una base de datos minimiza la duplicación de datos, y cuando la permite (por ejemplo, por rendimiento), lo hace de forma planeada y documentada, no accidental.

Integridad: el sistema aplica reglas (de dominio, de entidad, referenciales o del negocio) que garantizan que los datos almacenados sean válidos y consistentes entre sí.

Independencia de datos: los programas de aplicación quedan aislados de los detalles de cómo se almacenan físicamente los datos, de modo que cambios en el almacenamiento no obligan a reescribir las aplicaciones.

Seguridad: se controla quién puede leer o modificar cada porción de los datos, típicamente mediante cuentas, roles y permisos.

Concurrencia: el sistema gestor coordina el acceso simultáneo de múltiples usuarios o procesos, evitando que sus operaciones interfieran entre sí.

Recuperación: ante fallas de hardware, software o del sistema eléctrico, el gestor puede restaurar la base de datos a un estado consistente conocido, normalmente a partir de bitácoras y respaldos.

3. Archivos frente a bases de datos

Antes de los sistemas gestores, cada aplicación mantenía sus propios archivos, diseñados a la medida de esa aplicación. Este «enfoque de archivos» produce tres problemas estructurales (Elmasri & Navathe, 2017; Kroenke & Auer, 2016). El primero es la redundancia: si el dato de un cliente se necesita en el sistema de facturación y en el de envíos, cada uno mantiene su propia copia, duplicando el almacenamiento y el esfuerzo de captura. El segundo es la inconsistencia, consecuencia directa del anterior: cuando el mismo dato existe en varias copias, actualizar una y olvidar otra deja al sistema con versiones contradictorias de la misma realidad (por ejemplo, dos direcciones distintas para el mismo cliente). El tercero es la dependencia programa-datos: en un esquema de archivos, la estructura física del archivo está codificada dentro del programa que lo usa, de modo que un cambio aparentemente menor —agregar un campo, cambiar un tipo de dato— obliga a modificar y recompilar todos los programas que tocan ese archivo.

Estos problemas —no la simple acumulación de datos— son los que motivaron el desarrollo de los sistemas gestores de bases de datos: centralizar el almacenamiento, controlar la redundancia y desacoplar los programas de la representación física de los datos mediante una capa de independencia de datos.

4. Usuarios de una base de datos

Distintos roles interactúan con una base de datos, cada uno con responsabilidades y permisos propios (Silberschatz et al., 2020):

El diseñador de la base de datos identifica los datos que deben almacenarse y construye el modelo conceptual y lógico (por ejemplo, el diagrama entidad-relación) antes de que exista una sola línea de código de aplicación; su responsabilidad es que el esquema represente correctamente la realidad del negocio.

El administrador de bases de datos (DBA) tiene control central sobre el sistema: define el esquema físico, otorga y revoca permisos de acceso, gestiona el rendimiento, los respaldos y la recuperación ante desastres. Su nivel de permisos suele ser el más amplio de todos los roles.

El programador de aplicaciones escribe el software que usa la base de datos mediante un lenguaje de manipulación de datos embebido o una API; normalmente solo tiene los permisos necesarios sobre las tablas que su aplicación requiere, no acceso administrativo.

El usuario final interactúa con la base de datos indirectamente, a través de la interfaz de una aplicación (un punto de venta, un portal web), sin conocimiento del lenguaje de consulta ni del esquema subyacente; sus permisos están acotados a las operaciones que la aplicación expone.

5. Ciclo de vida de una base de datos

El desarrollo de una base de datos sigue, de forma análoga al ciclo de vida del software, una secuencia de etapas, cada una con un entregable concreto (Coronel & Morris, 2019; Elmasri & Navathe, 2017):

Planeación: se define el alcance y los objetivos del sistema de información; el entregable es un estudio de factibilidad.

Análisis de requerimientos: se entrevista a los usuarios y se documentan las necesidades de datos y de consulta; el entregable es una especificación de requerimientos.

Diseño conceptual: se construye un modelo independiente del gestor que se usará —típicamente un diagrama entidad-relación—; el entregable es el modelo conceptual validado con el cliente.

Diseño lógico: el modelo conceptual se traduce a las estructuras de un modelo de datos específico (por ejemplo, el modelo relacional), normalizando las relaciones; el entregable es el esquema lógico.

Diseño físico: se decide cómo se almacenarán físicamente los datos —índices, particiones, tipos de almacenamiento—; el entregable es el esquema físico y los scripts de creación.

Implementación y carga: se crea la base de datos en el gestor elegido y se pueblan las tablas, ya sea con datos históricos migrados o de prueba; el entregable es la base de datos operativa.

Prueba y evaluación: se valida que el sistema cumpla los requerimientos y se ajusta el rendimiento; el entregable es un reporte de pruebas.

Operación y mantenimiento: la base de datos entra en uso productivo y se le da mantenimiento continuo (respaldos, ajustes de esquema, monitoreo); el entregable son las bitácoras de operación y las actualizaciones del esquema a lo largo del tiempo.

6. Sistema gestor de bases de datos (SGBD)

Un sistema gestor de bases de datos es el software que media entre los usuarios/aplicaciones y la base de datos física, encargándose de definir, construir, manipular y compartir los datos de forma controlada (Elmasri & Navathe, 2017). Internamente se organiza en módulos como el procesador de consultas (que interpreta y optimiza las instrucciones), el gestor de almacenamiento (que traduce operaciones lógicas a accesos a disco), el gestor de transacciones (que garantiza atomicidad y aislamiento) y el gestor de bitácoras y recuperación.

El SGBD se opera mediante distintos sublenguajes (Silberschatz et al., 2020): el DDL (Data Definition Language) define la estructura —tablas, índices, restricciones—; el DML (Data Manipulation Language) permite insertar, consultar, actualizar y borrar datos; el DCL (Data Control Language) gestiona permisos (otorgar y revocar privilegios); y el control de transacciones (con instrucciones como COMMIT y ROLLBACK) delimita las unidades de trabajo que deben ejecutarse de forma atómica. Los SGBD se clasifican, entre otros criterios, por su modelo de datos (relacional, documental, de grafos, clave-valor, columnar), por el número de usuarios que soportan (mono-usuario o multiusuario) y por su distribución (centralizados o distribuidos).

7. Sistema de base de datos: arquitectura de tres niveles

La arquitectura ANSI/SPARC, ampliamente usada como referencia (Elmasri & Navathe, 2017; Date, 2004), organiza un sistema de base de datos en tres niveles. El nivel interno describe cómo se almacenan físicamente los datos (estructuras de archivos, índices, ubicación en disco). El nivel conceptual describe qué datos existen y cómo se relacionan, independientemente de cómo se almacenan o de quién los consulta; es la visión unificada de toda la organización. El nivel externo, por último, define vistas parciales adaptadas a distintos grupos de usuarios, mostrando solo la porción de datos relevante para cada uno.

Esta separación en niveles es precisamente la que sustenta la independencia de datos: la independencia lógica permite modificar el esquema conceptual (agregar una entidad, por ejemplo) sin afectar las vistas externas que no dependen de ese cambio; la independencia física permite modificar el almacenamiento (cambiar un índice, mover archivos a otro disco) sin afectar el esquema conceptual ni las aplicaciones. En cuanto a la arquitectura de despliegue, un sistema centralizado concentra el SGBD y los datos en un único servidor, mientras que una arquitectura cliente-servidor separa la lógica de presentación (clientes) de la gestión de datos (servidor), permitiendo que múltiples clientes remotos compartan un mismo servidor de base de datos a través de una red.

8. Modelos de datos

Un modelo de datos es un conjunto de conceptos que permite describir la estructura de una base de datos: qué tipos de datos existen, qué relaciones hay entre ellos y qué restricciones deben cumplir (Elmasri & Navathe, 2017). Su importancia radica en que sirve de puente entre la percepción del mundo real que tiene el usuario y la representación que finalmente entiende el SGBD, permitiendo razonar sobre el diseño antes de escribir una sola línea de código.

Los modelos de datos suelen clasificarse por su nivel de abstracción en tres categorías (Silberschatz et al., 2020): modelos conceptuales de alto nivel (como el modelo entidad-relación), que describen los datos cercanos a como los percibe el usuario, sin detalles de implementación; modelos lógicos basados en registros (como el relacional, el de red o el jerárquico), que describen los datos en términos de estructuras que el SGBD puede implementar directamente; y modelos físicos de bajo nivel, que describen cómo se almacenan los datos a nivel de archivos e índices. Otra clasificación habitual es por estructura: modelos basados en objetos (entidad-relación, orientado a objetos), modelos basados en registros (relacional, red, jerárquico) y, más recientemente, los modelos NoSQL (documental, clave-valor, columnar y de grafos), que relajan el esquema rígido del modelo relacional para escenarios de gran volumen o alta variabilidad estructural.

Referencias

- Coronel, C., & Morris, S. (2019). Database systems: Design, implementation, and management (13.^a ed.). Cengage Learning.
- Date, C. J. (2004). An introduction to database systems (8.^a ed.). Addison-Wesley.
- Elmasri, R., & Navathe, S. B. (2017). Fundamentals of database systems (7.^a ed.). Pearson.

Kroenke, D. M., & Auer, D. J. (2016). Database processing: Fundamentals, design, and implementation (14.^a ed.). Pearson.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database system concepts (7.^a ed.). McGraw-Hill.